
Rapport d'Alternance

Yani MOHELLEBI

Poste : Apprenti en Recherche & Développement en Intelligence Artificielle

Formation : Master 1 Informatique [BDIA](#)

Établissement : Université de Bourgogne Europe

Année universitaire : 2025 – 2026

Entreprise d'accueil : Siemens

Service : Digital Industries Software

Équipe : DI SW SIM AIDEC EEADDEV AIS

Adresse : 19 Bd Jules Carteret, 69007 Lyon

Tuteur entreprise : Bastien Lasteyrie

Contact entreprise : bastien.lasteyrie@siemens.com

Tutrice académique : Lylia Abrouk

Contact académique : lylia.abrouk@u-bourgogne.fr

Période d'alternance : 1^{er} octobre 2025 – 30 septembre 2027 (2 ans)

Projet présenté : AI GitLab Review

Résumé

Ce rapport présente AI GitLab Review, un outil d'analyse automatisée des merge requests développé durant mon alternance chez Siemens Digital Industries Software, à Lyon. Le projet vise à compléter la revue de code humaine en détectant, avant le relecteur, des défauts qui pourraient passer inaperçus.

La solution retenue repose sur une architecture multi-agents orchestrée par LangGraph : quatre agents spécialisés (sécurité, défauts logiques, gestion d'erreurs, qualité de code) analysent les modifications en parallèle, un classificateur distribue les fichiers aux agents concernés, et un synthétiseur déduplique les résultats par similarité sémantique avant de publier les commentaires sur GitLab via le Model Context Protocol. L'outil est aujourd'hui déployé auprès de quatorze équipes de développement, soit 103 utilisateurs actifs, et continue d'évoluer. Ce projet m'a permis d'acquérir des compétences en architecture logicielle, *prompt engineering*, orchestration d'agents [IA](#) et gestion d'un produit technique déployé auprès de quatorze équipes en production.

Mots-clés : revue de code, intelligence artificielle, [LLM](#), GitLab, système multi-agents, LangGraph, Model Context Protocol, déduplication sémantique, revue automatisée, R&D.

Table des matières

Résumé	1
Remerciements	4
Introduction	5
1 Présentation de l'entreprise	7
1.1 Siemens : présentation générale	7
1.2 Le département d'accueil	7
1.3 L'équipe	8
1.4 Environnement de travail et communication	9
2 Présentation des missions	11
2.1 Le poste d'apprenti R&D en Intelligence Artificielle	11
2.2 Contexte du projet AI GitLab Review	12
2.3 Éléments métier et fonctionnels	13
3 Analyse et conception	14
3.1 Analyse des solutions possibles	14
3.2 Démarche itérative et évolution du projet	15
3.3 Architecture	17
3.3.1 Architecture globale	18
3.3.2 Diagramme de classes	18
3.3.3 Pipeline de revue et machine à états LangGraph	20
3.3.4 Diagramme de séquence	20
3.3.5 Structure du projet	21
3.3.6 <i>Trust scoring</i> et <i>self-review</i>	22
3.3.7 Output structuré et <i>model tiering</i>	22
3.4 Gestion de projet	23
3.4.1 Méthodologie de développement	23
3.4.2 Règles de codage et qualité	23

3.4.3	Gestion des versions et intégration continue	24
4	Implémentation	25
4.1	Technologies et environnement de travail	25
4.1.1	Pourquoi TypeScript et Node.js	25
4.1.2	LangGraph : orchestration multi-agents	26
4.1.3	Model Context Protocol (MCP)	27
4.2	Développements réalisés	27
4.2.1	Les agents spécialisés	27
4.2.2	Le classificateur de diffs	31
4.2.3	La déduplication sémantique	32
4.2.4	Les outils de recherche pour agents	32
4.2.5	Les schémas Zod de <i>structured output</i>	32
4.3	Tracing et monitoring	33
4.3.1	LangSmith : tracing des appels LLM	33
4.3.2	Logging diagnostique	33
4.3.3	Langfuse : tracing complémentaire	33
4.4	Tests et validation	34
4.4.1	Stratégie de test	34
4.4.2	Tests unitaires (Vitest)	35
4.4.3	Couverture et parties non couvertes	35
4.4.4	Validation fonctionnelle	35
4.4.5	Limites connues	36
4.5	Résultats et perspectives	36
4.5.1	Résultats quantitatifs	36
4.5.2	Perspectives	37
	Conclusion	39
	Liste des acronymes	41
	Références bibliographiques	42

Remerciements

Je souhaite tout d'abord remercier **Bastien Lasteyrie**, mon tuteur entreprise, pour la qualité de son encadrement tout au long de cette alternance. Sa disponibilité, sa rigueur technique et la confiance qu'il m'a accordée dès les premières semaines m'ont permis de progresser rapidement et de mener le projet AI GitLab Review avec une réelle autonomie.

Je remercie **Lylia Abrouk**, ma tutrice académique, pour son suivi pédagogique et ses conseils tout au long de cette première année d'alternance.

Je remercie également mon manager, **Onur Atak**, et l'ensemble de l'équipe AIS pour la confiance qu'ils m'ont témoignée malgré la distance géographique, ainsi que les équipes du site de Lyon pour leur accueil chaleureux.

Enfin, je remercie **Mailys Nguyen**, *Talent Acquisition Specialist* du site de Lyon, pour son aide précieuse lors de mon intégration et les événements qu'elle organise avec les stagiaires et alternants.

Introduction

Avant qu'un développeur ne fusionne son code dans la branche principale d'un projet, un collègue le relit. C'est la revue de code, et c'est l'un des meilleurs moyens de détecter des défauts avant qu'ils n'atteignent la production. Le problème, c'est que cette pratique prend du temps (COHEN 2006). Et malgré cet effort, certaines failles de sécurité, certains défauts logiques ou certaines régressions de performance passent entre les mailles du filet.

Mon alternance se déroule chez **Siemens Digital Industries Software**, au sein de l'équipe AI Services sur le site de Lyon. En tant qu'apprenti R&D en intelligence artificielle, j'explore les technologies émergentes pour améliorer les processus d'ingénierie logicielle internes. Le premier projet qui m'a été confié, AI GitLab Review " désormais appelé AI Reviewer " vise à concevoir un outil capable d'assister les développeurs lors de la revue de leurs *merge requests*.

L'équilibre à trouver est délicat. L'outil doit repérer des problèmes que le relecteur pourrait manquer, que ce soit par fatigue ou par méconnaissance d'un domaine. Mais il ne doit pas générer de bruit : un commentaire non pertinent fait perdre du temps, et si cela arrive trop souvent, les développeurs finissent par ignorer l'outil. Tout le travail présenté dans ce rapport tourne autour de cette question :

Comment concevoir un système d'agents d'intelligence artificielle capable de compléter la revue de code humaine, tout en maintenant un taux de faux positifs suffisamment bas pour que les équipes l'adoptent ?

La figure 1 donne un aperçu concret du résultat : un commentaire généré automatiquement par AI Reviewer sur une merge request GitLab.

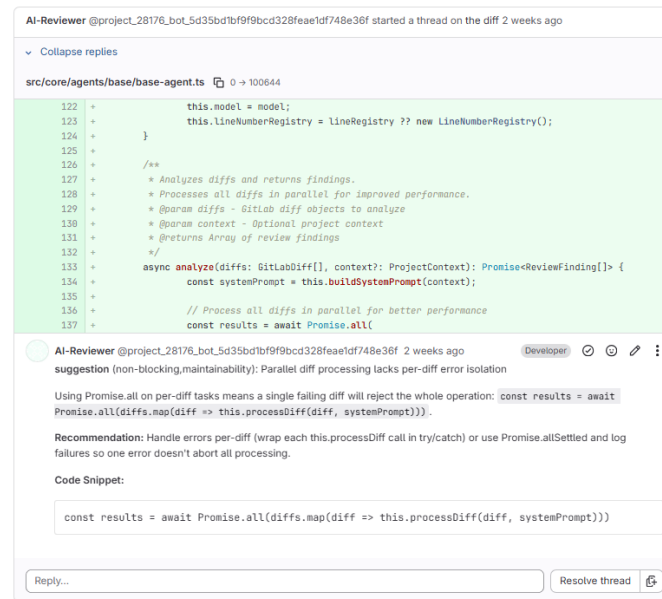


FIGURE 1 – Exemple de commentaire généré par AI Reviewer sur une merge request

Le rapport suit quatre chapitres. Le premier présente l'entreprise, l'équipe et l'environnement de travail. Le deuxième pose le contexte du projet, les objectifs et la démarche itérative suivie au fil des versions. Le troisième compare les solutions envisagées et décrit l'architecture retenue avec des diagrammes UML. Le quatrième porte sur l'implémentation, les tests, les difficultés rencontrées et les perspectives.

Chapitre 1

Présentation de l'entreprise

1.1 Siemens : présentation générale

Siemens AG, fondé en 1847 à Berlin, est l'un des plus grands groupes d'ingénierie au monde : environ 320 000 collaborateurs répartis dans plus de 200 pays, pour un chiffre d'affaires annuel de 75 à 83 milliards d'euros. Le siège social est à Munich.

Le groupe s'organise autour de trois entités principales : **Digital Industries** (automatisation et logiciel industriel), **Smart Infrastructure** (bâtiments et réseaux intelligents) et **Siemens Healthineers** (imagerie médicale). S'y ajoutent Siemens Mobility (transports ferroviaires) et Siemens Financial Services. Au fil des vingt dernières années, Siemens a cédé ses activités périphériques (semi-conducteurs, téléphonie, éclairage, énergie conventionnelle) pour se recentrer sur la numérisation de l'industrie.

C'est dans la branche Digital Industries, et plus précisément dans sa division logicielle, que s'inscrit mon alternance. Les logiciels développés par cette division servent des secteurs critiques (aéronautique, automobile, défense) où la qualité du code n'est pas optionnelle.

1.2 Le département d'accueil

Mon alternance se déroule au sein de **Siemens Digital Industries Software** (DI SW), la division logicielle de Digital Industries. Issue du rachat d'UGS en 2007, puis renforcée par les acquisitions de Mentor Graphics (2016) et Altair (2024), DI SW est l'un des premiers éditeurs mondiaux de logiciels industriels. Elle commercialise la plateforme **Xcelerator**, qui couvre l'ensemble du cycle de vie des produits : CAO (NX, Solid Edge), simulation (Simcenter), gestion du cycle de vie (Teamcenter), automatisation (Tecnomatix).

En France, DI SW emploie entre 500 et 999 salariés répartis sur plusieurs sites. Je suis basé au **19 boulevard Jules Carteret, 69007 Lyon**, sur un site qui héberge des équipes

de développement en simulation et en intelligence artificielle.

Je suis rattaché à l'équipe **DI SW SIM AIDEC EEADDEV AIS** (figure 1.1), qui travaille sur l'application de l'intelligence artificielle aux processus d'ingénierie logicielle. L'équipe est distribuée entre le Royaume-Uni, la Roumanie, la Belgique et la France. À Lyon, seuls mon tuteur Bastien Lasteyrie et moi-même sommes présents. Mon manager, Onur Atak, est au Royaume-Uni ; l'essentiel des échanges se fait à distance.



FIGURE 1.1 – Hiérarchie organisationnelle de l'équipe AIS au sein de Siemens

1.3 L'équipe

L'équipe AIS (*AI Services*) compte cinq membres répartis dans quatre pays :

- **Onur Atak** (Royaume-Uni) – manager de l'équipe.
- **Bastien Lasteyrie** (France, Lyon) – mon tuteur entreprise, orienté *front-end* et infrastructure [CI/CD](#).
- **Kefan Sun** (Belgique) – *prompt engineering*, évaluation de modèles et agents.
- **Ozgur Yuksel** (Roumanie) – *back-end* et intégration de services.
- **Yani Mohellebi** (France, Lyon) – moi-même, apprenti R&D en intelligence artificielle.

La mission de l'équipe est de faire le lien entre les équipes de développement de Siemens DI SW et les technologies d'intelligence artificielle émergentes : accompagnement à l'intégration de chatbots, évaluation de modèles, conception de *workflows* agentiques et développement d'outils internes .

Mes interlocuteurs directs sont Bastien (encadrement technique quotidien) et toute l'équipe.

1.4 Environnement de travail et communication

Organisation interne de l'équipe AIS

L'équipe AIS fonctionne selon une cadence agile. Chaque matin à 10 heures, un *daily meeting* de trente minutes réunit l'ensemble des membres par visioconférence : chacun résume le travail accompli la veille et annonce ses objectifs du jour. Ce rituel court permet de détecter rapidement les blocages et de maintenir la cohérence malgré la répartition géographique. Toutes les trois semaines, deux cérémonies structurent le cycle de développement : un *refinement*, consacré à la clarification et à l'estimation des prochaines tâches, et une *retrospective*, dédiée à l'analyse de ce qui a fonctionné ou non durant le sprint écoulé.

L'organisation des tâches repose sur **Jira**, où chaque *user story*, *bug* ou *spike* est tracé, priorisé et assigné (figure 1.2). La documentation technique, les résultats de recherche et les ressources partagées sont centralisés sur **Confluence**, ce qui garantit la traçabilité des décisions et facilite l'intégration de nouveaux membres. La communication quotidienne transite par **Microsoft Teams**, que ce soit pour les messages instantanés, les appels ou le partage de fichiers.

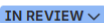
 STSAIS-835	Skip diffs with no added lines and resolve contextProvider issue	 N...	0	 MA	M..	 CLOSED
 STSAIS-836	feat :add searchRepository and fileContext tools for the agents in AI reviewer	 N...	0	 MA	M..	 IN REVIEW
 STSAIS-837	Expose file_context MCP tool as a LangChain DynamicStructuredTool	 Hi...	5	 MA	M..	 CLOSED
 STSAIS-838	Expose search_repository MCP tool as LangChain DynamicStructuredTool	 Hi...	5	 MA	M..	 CLOSED
 STSAIS-840	Inject MR description into agent system prompt	 N...	0	 Una		 BACKLOG

FIGURE 1.2 – Exemple de tâches du projet AI Reviewer dans Jira

Communication avec les équipes utilisatrices

AI Reviewer est déployé auprès de quatorze équipes de développement au sein de Siemens DI SW. Pour structurer les échanges avec ces équipes, un channel Teams dédié « AIS-AI Reviewer » regroupe 103 membres, organisé en trois fils de discussion distincts. Le premier, *Announcements*, sert à diffuser les notes de version lors de chaque nouvelle release (figure 1.3). Le deuxième, *Feedback and Support*, centralise les retours d'expérience et les signalements d'incidents : en rendant les réponses visibles par tous, chaque équipe bénéficie des solutions apportées aux autres. Le troisième, *Getting Started*, accompagne les nouveaux utilisateurs dans l'intégration d'AI Reviewer dans leur *workflow*, avec des étapes détaillées et des liens vers des pages Confluence en cas de difficulté.

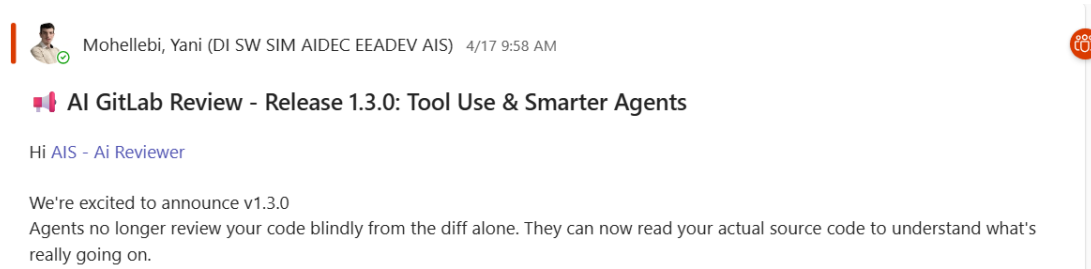


FIGURE 1.3 – Exemple d’annonce de release sur le channel Teams AI Reviewer

Lorsqu’une équipe souhaite approfondir le fonctionnement de l’outil, demander une nouvelle fonctionnalité, signaler un problème plus complexe ou contribuer au projet, nous organisons directement un appel avec mon tuteur. Nous avons ainsi eu un échange dédié avec la quasi-totalité des équipes utilisatrices, parfois davantage. Ce contact direct permet de recueillir des besoins concrets et de construire une relation de confiance avec les utilisateurs.

Vie de site et formation continue

Le site de Lyon offre un cadre propice aux échanges : je participe régulièrement aux événements informels en soirée, et j’y présente mes projets. Siemens met également à disposition une **Global Student Community** internationale et impose un minimum de vingt heures de *Learning Days* par an, que j’investis dans des sujets alignés avec mes missions : nouvelles technologies en [IA](#), GitHub Copilot, Langfuse, ou encore des formations transverses (*Digital Transformation, Business of Engineering*) sur les plateformes internes.

Chapitre 2

Présentation des missions

2.1 Le poste d'apprenti R&D en Intelligence Artificielle

À ce poste de R&D en intelligence artificielle, mon rôle consiste à explorer les technologies émergentes en [IA](#) et à les mettre au service des équipes d'ingénierie logicielle de Siemens.

Veille technologique et partage de connaissances

Le domaine de l'intelligence artificielle évolue à un rythme soutenu : nouveaux modèles de langage, frameworks d'orchestration, protocoles d'intégration. Dans ce contexte, une part importante de mon travail consiste à suivre les annonces des principaux éditeurs (OpenAI, Anthropic, Google), à lire leur documentation technique et à évaluer les outils qui pourraient bénéficier au projet. Lorsqu'une technologie se révèle pertinente, je rédige une synthèse sur Confluence afin que l'équipe puisse en tirer parti sans refaire le même travail de recherche.

Mes collègues, plus expérimentés dans leurs domaines respectifs, constituent une aide précieuse au quotidien : Kefan sur le *prompt engineering*, Bastien sur le TypeScript, le [CI/CD](#) GitLab et Docker.

Développement logiciel

Le développement représente la majeure partie de mon temps. Depuis le début de l'alternance, l'ensemble de mon travail de code porte sur AI GitLab Review.

Au fil des mois, j'ai adopté un cycle de développement itératif : identifier un problème, poser l'architecture, implémenter sur une branche dédiée, tester sur des [MR](#) réelles, puis décider de conserver ou d'abandonner la piste. Ce cycle, parfois très court (certaines pistes ont été écartées en une journée), est détaillé en section [3.2](#).

Le projet mobilise plusieurs domaines techniques que j’ai dû apprendre ou approfondir en parallèle : architecture logicielle (*design patterns*, séparation des responsabilités), *prompt engineering* (rédaction d’instructions pour les modèles, règles anti-hallucination, *structured output*), orchestration d’agents avec LangGraph (graphe d’état, parallélisme, routage), validation de schémas avec Zod (MCDONNELL 2024), et intégration d’API (Azure OpenAI (MICROSOFT 2024), GitLab via MCP, LangSmith (LANGCHAIN, INC. 2024c) pour le tracing).

Support et déploiement

AI Reviewer étant utilisé en production, j’assure également le support auprès des équipes utilisatrices (section 1.4). Ce volet m’a conduit à me former à Docker, aux fichiers `.gitlab-ci.yml`, aux registres d’images et à la gestion des variables d’environnement dans les pipelines.

Autonomie et encadrement

Le poste implique une autonomie importante au quotidien : je choisis les technologies à explorer, je conçois l’architecture et je planifie mon travail. Bastien encadre cette autonomie en relisant l’ensemble de mon code, en confrontant mes choix techniques et en me signalant les approches qui ne tiennent pas la route.

2.2 Contexte du projet AI GitLab Review

Au sein de Siemens DI SW, toute modification de code passe par une [Merge Request \(MR\)](#) sur GitLab. Un collègue relit les changements, rédige ses remarques et valide ou refuse la [MR](#). Cette pratique est essentielle dans un contexte où les logiciels servent des secteurs critiques, mais elle repose entièrement sur le travail humain. Selon la taille de la [MR](#), la relecture peut prendre de vingt minutes à plusieurs heures.

Quand Bastien m’a présenté le projet, j’ai voulu comprendre ce qui rendait la revue de code difficile avant d’aborder la partie technique. J’ai consulté plusieurs ressources sur le sujet, dont l’étude de SmartBear menée chez Cisco sur environ 2 500 revues de code (COHEN 2006). Les résultats sont sans ambiguïté : au-delà de 200 à 400 lignes modifiées, le taux de détection de défauts chute fortement. Le relecteur se fatigue, survole le code et laisse passer des erreurs. L’étude indique également qu’une relecture efficace ne devrait pas excéder 60 à 90 minutes. À cela s’ajoute le problème de l’expertise : un relecteur ne couvre pas tous les domaines, et deux relecteurs ne formuleront pas les mêmes remarques.

Ces lectures m’ont permis de cadrer l’ambition du projet : produire une revue automatisée qui se rapproche le plus possible d’une relecture humaine. Un simple outil qui se contente d’envoyer le diff à un modèle et de relayer sa réponse ne suffit pas ; il faut que l’outil

comprenne le contexte du code, formule des remarques pertinentes et les positionne au bon endroit. Cette ambition soulève plusieurs contraintes techniques : gestion du contexte limité des LLM, maîtrise du coût en tokens, réduction des faux positifs et maintien d’une latence compatible avec un pipeline CI.

Le projet s’inscrit par ailleurs dans un cadre organisationnel contraint : seul Azure OpenAI est autorisé (pas d’API publique), et le code source, propriétaire, ne doit transiter par aucun service non approuvé.

Du point de vue de l’utilisateur, le fonctionnement est simple : le développeur ouvre sa MR sur GitLab et lance l’analyse en cliquant sur un bouton (figure 2.1). Les commentaires générés apparaissent directement dans la MR, au même endroit que les commentaires humains.



FIGURE 2.1 – Lancement d’une revue AI Reviewer depuis l’interface GitLab

2.3 Éléments métier et fonctionnels

Le vocabulaire du projet repose sur cinq termes :

- **Merge Request (MR)** : ensemble de modifications soumises à relecture sur GitLab.
- **Diff** : changements d’un fichier au sein d’une MR.
- **Hunk** : bloc contigu de lignes modifiées au sein d’un diff. C’est l’unité d’analyse de l’outil.
- **Finding** : constat émis par un agent (titre, description, extrait de code). Peut inclure un correctif suggéré.
- **Label** : étiquette issue de la norme *Conventional Comments* (SHORTINO 2020) (issue, suggestion, nitpick ou praise).

Chapitre 3

Analyse et conception

3.1 Analyse des solutions possibles

À mon arrivée, le projet reposait sur un agent unique avec un seul prompt (prototype Python, puis réécriture TypeScript (LANGCHAIN, INC. 2024a)). Mon travail a consisté à faire évoluer cette base vers un système capable de détecter des défauts invisibles dans le diff seul, tout en maintenant un taux de faux positifs suffisamment bas pour que les équipes adoptent l'outil. Trois approches ont été envisagées :

1. **Monolithique** : un seul appel LLM avec l'intégralité du diff. Testée deux fois, écartée.
2. **Multi-agents LangGraph** : agents spécialisés orchestrés par LangGraph (LANGCHAIN, INC. 2024b), avec routage *per-hunk*. Retenue.
3. **GitHub Copilot SDK** : intégration du runtime agentique de Copilot CLI dans l'application via JSON-RPC (GITHUB, INC. 2026). Annoncé début 2026, en *public preview* depuis.

Le tableau 3.1 compare ces trois approches sur les critères déterminants pour le projet.

TABLE 3.1 – Comparaison des trois approches de revue automatisée

Critère	Monolithique	Multi-agents LangGraph (re- tenu)	GitHub Copilot SDK
Architecture	1 prompt, 1 appel	Agents spécialisés + classificateur + synthétiseur	Runtime agentique Copilot CLI (boîte noire)
Précision	Perte de focus sur les diffs longs	Prompts spécialisés par domaine	Dépend des capacités du runtime
Faux positifs	Élevés, pas de filtrage	<i>Trust scoring + self-review</i>	Pas de mécanisme intégré
Coût / MR	~\$0,01	~\$0,02 (négligeable vs. temps humain)	Quota <i>premium requests</i> Copilot
Parallélisme	Aucun	<i>Fan-out</i> natif LangGraph	Géré par le runtime (non configurable)
<i>Structured output</i>	Parsing JSON manuel	Natif via Zod + LangChain	Non exposé
Gestion des outils	Manuelle	Intégrée (helper <code>tool()</code> + schéma Zod)	Outils prédéfinis (fichiers, Git, web)
Tracing / debug	Aucun	LangSmith natif	Logs CLI uniquement
Extensibilité	Fragile	1 agent + 1 prompt par domaine	<i>Skills</i> et outils custom, mais runtime opaque

Depuis le début du projet, deux outils majeurs de revue par IA sont apparus sur le marché. **GitHub Copilot code review**, disponible en accès général début 2025, permet de demander une revue automatique sur une *pull request* GitHub. **Claude Code Review** (ANTHROPIC 2025), proposé par Anthropic en *research preview* pour les abonnements Team et Enterprise, repose sur une flotte d’agents spécialisés qui analysent le diff en parallèle.

L’émergence de ces outils valide l’approche, mais aucun ne répond aux contraintes de Siemens : le code source propriétaire ne peut pas transiter par un service externe non approuvé. Un outil interne, déployé sur Azure, offre un contrôle total sur les modèles, les catégories de détection, les prompts et la confidentialité des données.

3.2 Démarche itérative et évolution du projet

Le développement d’AI Reviewer n’a pas suivi de plan figé. Chaque version a été construite en réponse à un problème concret, testée sur des MR réelles, puis conservée ou abandonnée

en fonction des résultats obtenus.

Fondations et spécialisation (sept.–nov. 2025). Le projet a débuté par le choix d’un framework d’orchestration. J’ai d’abord évalué CrewAI (CREWAI, INC. 2024) pendant deux jours, avant de l’écarter en raison de son manque de flexibilité : pas de routage conditionnel, pas de parallélisme. LangGraph (LANGCHAIN, INC. 2024b) répondait mieux au besoin : il offre un graphe d’état explicite avec *fan-out* dynamique. Une fois l’architecture posée avec le MCP et le pipeline CI, j’ai découpé l’agent monolithique en cinq agents spécialisés autour d’une classe abstraite `BaseAgent`, ajouté un synthétiseur pour fusionner les résultats, et migré le déploiement vers Docker. La figure 3.1 présente cette première architecture.

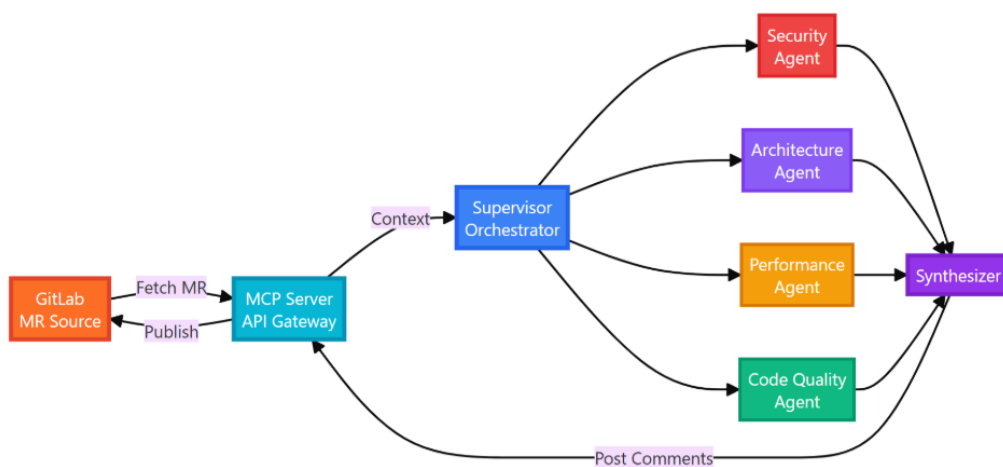


FIGURE 3.1 – Architecture initiale : orchestrateur, agents spécialisés et synthétiseur simple

Fiabilisation et classification (nov. 2025–jan. 2026). Le principal défaut de cette première version résidait dans la duplication : chaque diff était transmis à l’ensemble des agents, même lorsque cela n’était pas pertinent. En conséquence, plusieurs agents produisaient des remarques identiques sur le même code. Pour y remédier, j’ai introduit un `ClassifierAgent` chargé de router chaque fichier vers les agents concernés, ainsi qu’un mécanisme de déduplication par embeddings pour fusionner les findings similaires. J’ai également intégré Zod (MCDONNELL 2024) pour valider la sortie structurée des agents, créé un registre de numéros de ligne pour positionner correctement les commentaires dans GitLab, et écrit les premiers tests unitaires. Afin de valider l’intérêt de la spécialisation, j’ai testé un agent monolithique unique (*Big Agent*) dont les résultats se sont révélés nettement inférieurs. Deux agents expérimentaux (Typos, Recommendation) ont été testés puis abandonnés. Cette étape a abouti à la première version stable (tags 1.0.x, puis 1.1.0).

Granularité et outillage (jan.–avr. 2026). Le classificateur raisonnait alors par fichier, ce qui posait un problème de granularité : un fichier relevant à la fois de la sécurité

et de la gestion d'erreurs était soit routé vers un seul agent, soit envoyé en entier aux deux. Le passage à la classification *per-hunk* a résolu cette difficulté : chaque fichier est découpé en hunks, et chaque hunk est routé indépendamment vers l'agent concerné. Ce découpage a également corrigé le positionnement des commentaires inline (tags 1.2.x). En parallèle, j'ai ajouté les labels *Conventional Comments*, amélioré les prompts, donné aux agents un accès au codebase via trois outils [MCP](#), intégré Codex pour la génération de correctifs, et mis en place la détection de commentaires dupliqués. L'idée d'un Verification Agent séparé a été explorée, puis absorbée dans le mécanisme de *trust scoring*. Tags 1.2.2, puis 1.3.0 (version en production). La figure 3.2 présente l'architecture à ce stade.

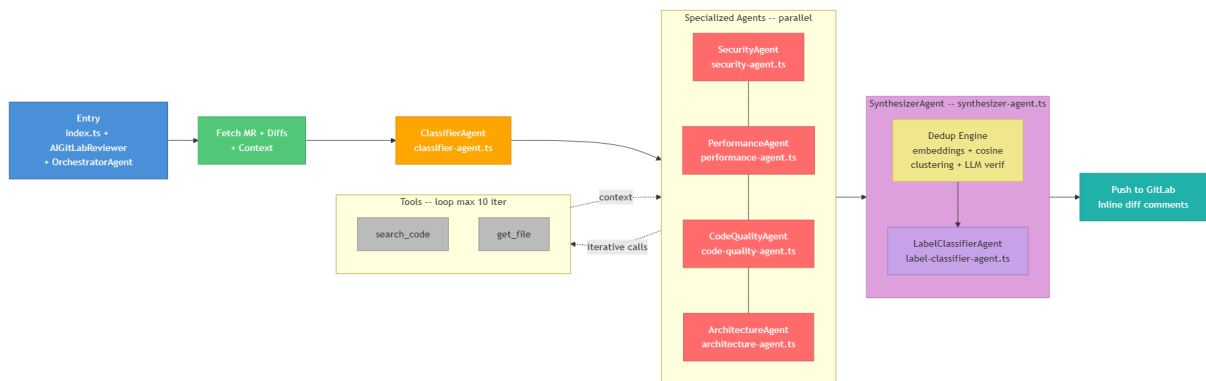


FIGURE 3.2 – Architecture v1.3 : classificateur *per-hunk*, outils [MCP](#), synthétiseur et déduplication

Refactoring architectural (avr. 2026—en cours). La sortie de LangChain 1.x a été le déclencheur de cette phase. La nouvelle version stable du framework apporte des améliorations directement exploitables : intégration native des modèles Azure OpenAI plus robuste, gestion simplifiée des outils, et support natif du *structured output* Zod sans couche d'adaptation. Plutôt que de corriger le code existant de manière incrémentale, j'ai choisi de reconstruire l'arborescence en séparant clairement les responsabilités. Ce refactoring est bien avancé.

3.3 Architecture

Cette section décrit l'architecture issue du refactoring, celle sur laquelle le développement se poursuit. Elle reprend les principes validés en production (pipeline séquentiel, agents parallèles, déduplication) tout en les restructurant grâce aux apports de LangChain 1.x (*structured output* Zod natif, gestion standardisée des outils, intégration Azure OpenAI renforcée).

3.3.1 Architecture globale

Le pipeline de revue s'organise en six nœuds exécutés séquentiellement par le graphe d'état LangGraph :

1. `fetch_mr` : récupération des métadonnées de la [MR](#) via [MCP](#) ;
2. `fetch_diffs` : récupération des diffs (fichiers modifiés) ;
3. `analyze_context` : extraction du contexte projet (titre, description) ;
4. `classify_diffs` : classification *per-hunk*, routage vers les agents concernés ;
5. `coordinate_agents` : exécution parallèle des agents spécialisés via *Send fan-out* ;
6. `finalize` : déduplication sémantique des findings et publication sur GitLab.

Les quatre agents de détection (section [4.2.1](#)) s'exécutent en parallèle à l'étape 5. Le synthétiseur, à l'étape 6, fusionne les résultats et publie les commentaires via le [MCP](#) GitLab.

3.3.2 Diagramme de classes

La hiérarchie des agents repose sur une classe abstraite `BaseAgent` qui définit le squelette d'analyse via le pattern *Template Method* : chaque agent concret implémente sa propre méthode `buildPrompt()` tout en héritant du flux commun (appel [LLM](#), parsing de la réponse, extraction des findings).

Les classes de support suivent des patterns classiques : `AgentFactory` (Factory Method) instancie les agents par catégorie, `AgentRegistry` offre un accès $O(1)$ par clé, `GitLabMCPWrapper` (Facade) encapsule les appels [MCP](#), et `OpenAIFactory` centralise la création des instances Azure OpenAI.

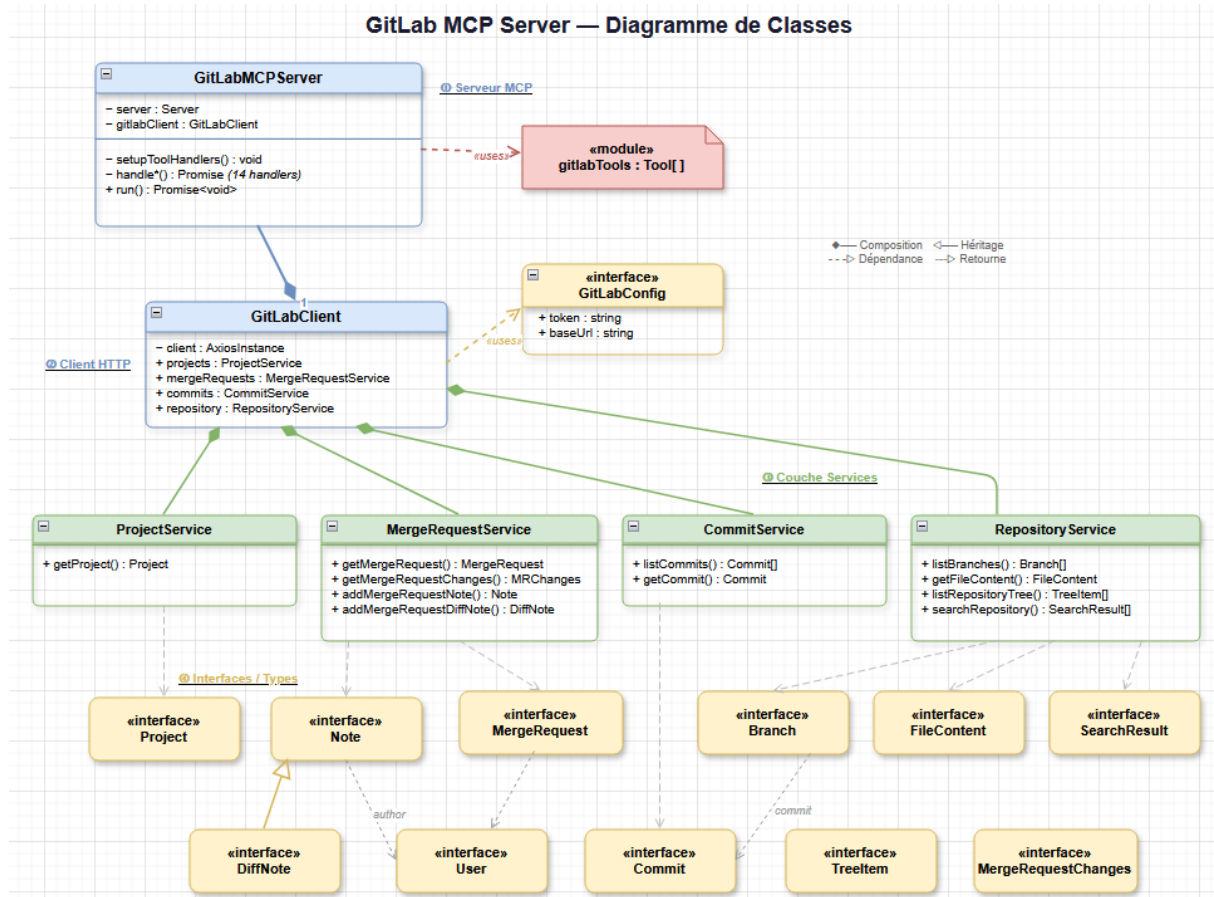
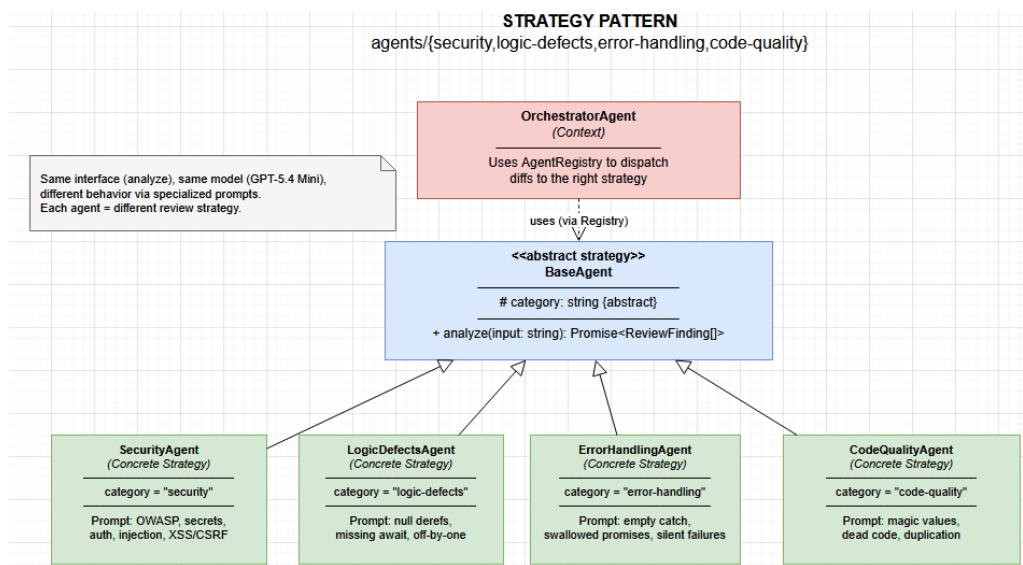


FIGURE 3.3 – Diagramme de classes : hiérarchie des agents et modules de support

Chaque agent concret constitue une stratégie interchangeable (pattern *Strategy*) : l'orchestrateur sélectionne les agents à exécuter en fonction des catégories retournées par le classificateur, sans connaître leur logique interne. La figure 3.4 détaille cette organisation.

FIGURE 3.4 – Pattern *Strategy* : sélection dynamique des agents par catégorie

3.3.3 Pipeline de revue et machine à états LangGraph

LangGraph modélise le pipeline comme un `StateGraph` typé : chaque nœud est une fonction pure qui reçoit l'état courant et retourne un état modifié. Les transitions sont explicites (pas de décision par LLM), ce qui rend le flux déterministe et testable.

Le parallélisme repose sur le pattern *Map-Reduce* (figure 3.5) : la phase *map* distribue les hunks aux agents via `Send`, et la phase *reduce* agrège les findings avant déduplication.

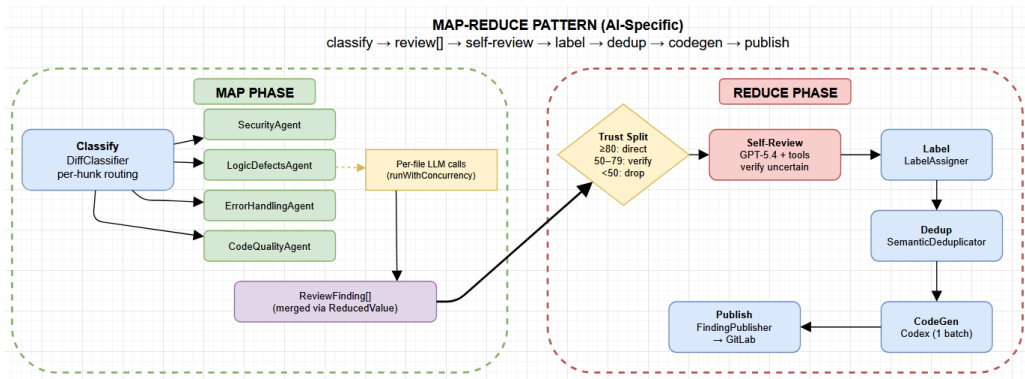


FIGURE 3.5 – Pattern *Map-Reduce* : distribution parallèle des hunks et agrégation des findings

3.3.4 Diagramme de séquence

Le flux complet d'une revue se déroule ainsi : le pipeline `CI` GitLab déclenche l'outil, qui récupère la `MR` et ses diffs via `MCP`, les classe par catégorie, lance les agents en parallèle, déduplique les findings, puis publie les commentaires inline dans l'interface GitLab. L'ensemble s'exécute en cinq minutes en moyenne.

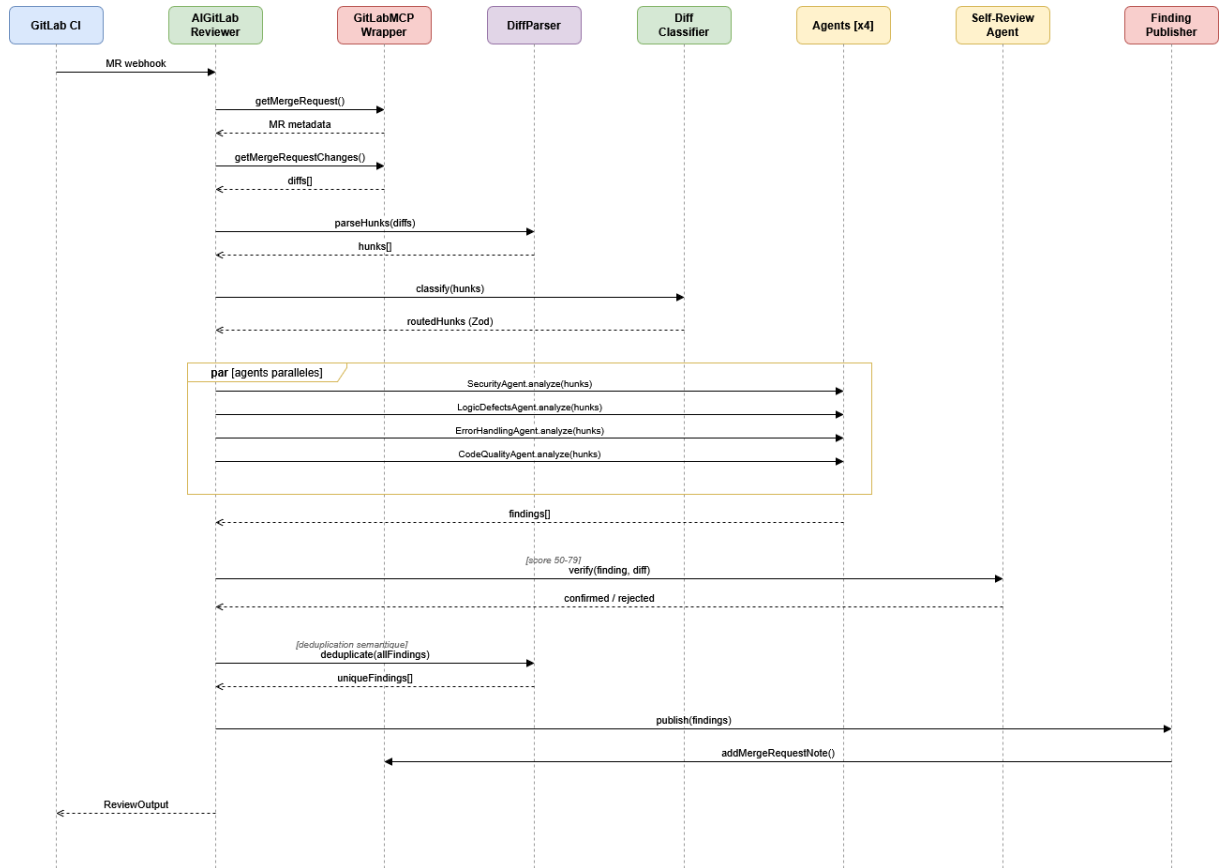


FIGURE 3.6 – Diagramme de séquence : flux complet d'une revue, du déclenchement CI à la publication

3.3.5 Structure du projet

L'arborescence cible organise le code en modules à responsabilité unique :

- `src/agents/` : les quatre agents de détection, l'orchestrateur LangGraph et le registre/factory d'agents ;
- `src/review/` : le pipeline de traitement (parsing des diffs, findings, déduplication, classifier, publisher) ;
- `src/gitlab/` : la facade [MCP](#) (`GitLabMCPWrapper`), seul point de contact avec l'[API](#) GitLab ;
- `src/llm/` : la factory OpenAI, l'initialisation des embeddings et le chargement des prompts ;
- `src/tools/` : les trois outils de recherche exposés au *self-review agent* ;
- `src/config/constants/` : les constantes métier, réparties par domaine (modèles, findings, déduplication, etc.) ;
- `src/prompts/` : les fichiers Markdown des *prompts*, un par agent/service ;
- `src/types/` : les schémas *Zod* de *structured output* et les interfaces partagées ;
- `src/utils/` : les helpers transversaux (logger, gestion d'erreurs, limiteur de concur-

rence);

- `src/cli/` : les règles de diagnostic et l’affichage des résultats en mode CLI.

Chaque module est conçu pour être testable indépendamment.

3.3.6 *Trust scoring et self-review*

Le *trust scoring* est le mécanisme central de réduction des faux positifs dans l’architecture cible. Chaque agent de détection attribue à ses findings un score de confiance entre 0 et 100, selon trois paliers :

- ≥ 80 : le finding est prouvable à partir du diff seul. Il est publié directement.
- **50–79** : le finding est suspect mais non confirmé. Il est transmis au self-review agent (GPT-5.4 avec accès au codebase via outils) pour vérification. Si confirmé, le score est promu ; sinon, le finding est écarté.
- < 50 : le finding est trop spéculatif. Il est écarté sans vérification.

Ces seuils sont des heuristiques calibrées empiriquement sur un échantillon de findings annotés manuellement. Ils sont appelés à être affinés à mesure que le dataset d’évaluation interne s’étoffe.

Le *self-review agent* reçoit le finding et le diff original, mais pas le raisonnement de l’agent de détection, afin d’éviter le biais de confirmation. Ce principe de *fresh context* force le modèle à former son propre jugement. La figure 3.7 illustre ce flux de vérification.

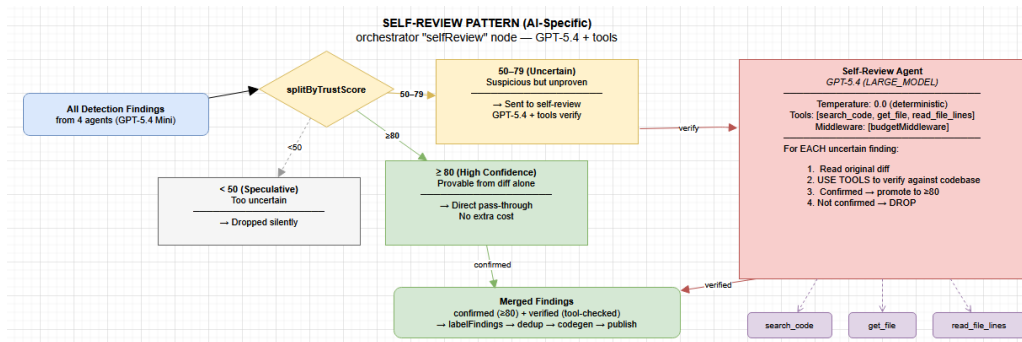


FIGURE 3.7 – Pattern *Self-Review* : vérification indépendante des findings par un second modèle

3.3.7 Output structuré et *model tiering*

Chaque étape du pipeline qui fait appel à un LLM utilise un schéma Zod pour contraindre la sortie. Le modèle ne peut pas produire de réponse hors format : si un champ est manquant ou d’un mauvais type, LangChain relance l’appel.

Le *model tiering* répartit les appels entre trois modèles Azure OpenAI en fonction du rapport coût/capacité :

- **GPT-5.4 Mini** (détection et classification) : modèle rapide et économique, utilisé pour les appels à haut volume (quatre agents $\times$ N hunks).
- **GPT-5.4** (*self-review*) : modèle plus puissant avec accès aux outils, réservé aux findings incertains (50–79).
- **Codex** (génération de correctifs) : modèle spécialisé code à température 0, appelé en batch sur les findings finaux.

Ce découpage permet de maîtriser le coût tout en réservant la puissance de raisonnement aux étapes qui en ont besoin. La figure 3.8 illustre ce principe de séparation des responsabilités entre les trois modèles.

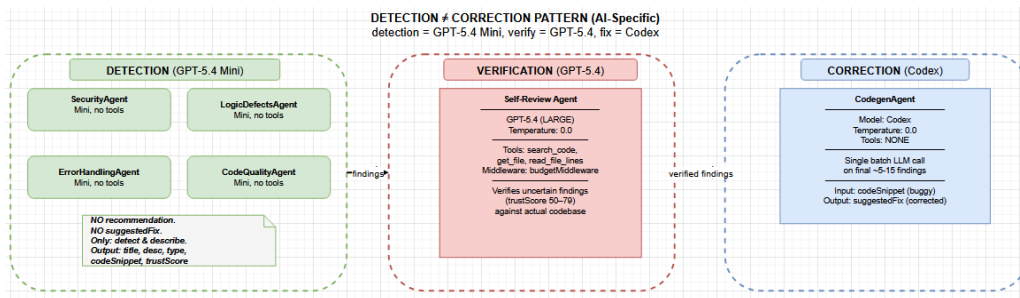


FIGURE 3.8 – Pattern détection $\neq$ correction : répartition des responsabilités entre modèles

3.4 Gestion de projet

3.4.1 Méthodologie de développement

Le cadre agile de l'équipe AIS (sprints de trois semaines, *daily meetings*, rétrospectives) a été décrit dans la section 1.4. Pour le projet AI Reviewer en particulier, le développement suit une approche itérative décrite dans la section 3.2 : chaque version répond à un problème concret, est testée sur des MR réelles, et est conservée ou abandonnée selon les résultats.

Le suivi des tâches passe par Jira (*user stories*, *bugs*, *spikes*). Le refactoring en cours est planifié sous forme d'un *roadmap* de MR.

3.4.2 Règles de codage et qualité

Des conventions TypeScript strictes encadrent le développement :

- Indentation par tabulations, pas d'espaces ;
- Pas de type `any` : utiliser `unknown` avec un type guard ;
- `as const` sur tous les objets de configuration ;

- *Zéro magic strings/numbers* : toute valeur littérale à signification métier est extraite dans `src/config/constants/` ;
- *Early return* systématique, pas d’imbrication conditionnelle au-delà de trois niveaux.

ESLint vérifie ces règles à chaque commit.

3.4.3 Gestion des versions et intégration continue

Le développement suit un *workflow* Git basé sur des branches *feature* : chaque fonctionnalité ou correctif est développé sur une branche dédiée, soumis via une [MR](#), relu par mon tuteur (et par AI Reviewer lui-même), puis fusionné dans la branche principale. Les versions stables sont marquées par des tags sémantiques (1.0.0 à 1.3.0).

Le pipeline [CI/CD](#) GitLab exécute quatre étapes : compilation TypeScript, exécution des tests Vitest, vérification ESLint, puis construction de l’image Docker. L’image est poussée dans le registre d’images GitLab du projet, d’où les équipes utilisatrices la récupèrent pour l’intégrer dans leur propre pipeline.

Chapitre 4

Implémentation

4.1 Technologies et environnement de travail

Le tableau 4.1 récapitule les technologies utilisées dans le projet. Chaque choix répond à un besoin précis du pipeline : TypeScript pour le typage statique et la compatibilité avec l'écosystème LangChain, LangGraph pour l'orchestration multi-agents, Zod pour contraindre les sorties des LLM, et le MCP pour isoler les appels GitLab. Les sous-sections suivantes détaillent les choix les plus structurants.

TABLE 4.1 – Stack technique du projet

Catégorie	Technologie	Rôle dans le projet
Langage	TypeScript 5.x	Typage statique, maintenabilité
Runtime	Node.js 20+	Environnement d'exécution côté serveur
Orchestration	LangGraph	Graphe d'état multi-agents avec <i>Send fan-out</i>
LLM (détection)	GPT-5.4 Mini	Détection rapide et peu coûteuse (temp. 0.1)
LLM (vérification)	GPT-5.4	Self-review avec outils (temp. 0)
LLM (codegen)	Codex	Génération de correctifs (temp. 0.0)
GitLab	MCP (studio)	Interface standardisée avec l'API GitLab
Validation	Zod	Schémas <i>structured output</i> + validation runtime
Tests	Vitest	Tests unitaires avec mocks (<code>vi.fn()</code>)
Conteneur	Docker	Déploiement reproductible
Lint	ESLint	Application des règles de codage
Monitoring	LangSmith	Tracing des appels LLM et debugging

4.1.1 Pourquoi TypeScript et Node.js

Le choix de TypeScript et Node.js a été fait par mon tuteur lors du prototype initial du projet. Ce choix s'est révélé pertinent pour plusieurs raisons :

- **Écosystème IA mature** : LangChain.js et LangGraph disposent d’une documentation complète et d’un support actif en TypeScript, rendant l’intégration des [LLM](#) directe.
- **Typage statique** : le typage de TypeScript garantit la maintenabilité à long terme, en particulier sur les schémas de *structured output* (Zod) et les interfaces entre modules.
- **Compatibilité MCP** : le serveur MCP GitLab officiel est lui-même écrit en TypeScript, ce qui permet une intégration native sans couche d’adaptation.
- **Maintenabilité** : les conventions strictes (types explicites, `as const`, *early return*) rendent le code lisible et auditable par d’autres développeurs de l’équipe.

4.1.2 LangGraph : orchestration multi-agents

Le projet utilise LangGraph pour orchestrer le pipeline multi-agents. LangGraph est une extension de LangChain qui modélise les flux sous forme de *StateGraph*, une machine à états explicite où chaque nœud correspond à une étape du pipeline (le choix de LangGraph par rapport aux alternatives est détaillé en section 3.2).

Le développement a démarré sur LangGraph 0.2.x, qui nécessitait des contournements manuels (concaténation d’arrays dans les reducers, `Promise.all` pour le parallélisme, routage conditionnel par chaînes de caractères). Le refactoring en cours migre vers la version 1.3, qui apporte des améliorations structurantes :

TABLE 4.2 – Évolution des primitives LangGraph entre 0.2.x et 1.3

Primitive	Ancien (0.2.x)	Nouveau (1.3)
Définition d’état	<code>Annotation.Root({})</code>	<code>StateSchema</code> avec validation Zod
Accumulateur	Concat manuel dans le reducer	<code>ReducedValue(schema, {reducer})</code>
Fan-out	<code>Promise.all</code> dans un nœud	<code>Send("node", payload)</code> – état par worker, traçable
Routage conditionnel	<code>addConditionalEdges</code> (strings)	<code>Command({goto})</code> typé
Typage des nœuds	Cast manuel	<code>GraphNode<typeof State></code>
Checkpointing	Externe	<code>MemorySaver</code> intégré
Interruptions	N/A	<code>interrupt(payload)</code> – pause pour approbation humaine
Sous-graphes	Câblage manuel	Propagation mémoire automatique

4.1.3 Model Context Protocol (MCP)

Le [Model Context Protocol \(MCP\)](#), proposé par Anthropic en 2024, est un protocole standardisé qui permet à un agent [IA](#) d'interagir avec des systèmes externes via un ensemble d'outils déclarés. Le transport utilisé ici est *stdio* : le serveur MCP s'exécute comme un processus enfant de l'application, communiquant par entrée/sortie standard en JSON-RPC. Ce choix isole complètement la couche I/O GitLab du reste du pipeline : les agents ne connaissent ni l'[API REST](#) GitLab, ni ses endpoints, ni son authentification.

Le serveur MCP GitLab est un package initié par mon tuteur, que j'ai ensuite enrichi avec des outils supplémentaires (`@sts-ai-services/gitlab-mcp`). Il expose douze outils répartis en trois domaines : gestion des merge requests (lecture, commentaires inline), accès au repository (contenu de fichiers, recherche dans le code) et consultation des commits.

En pratique, le pipeline de revue n'utilise qu'un sous-ensemble de ces outils : `get_merge_request_changes` pour récupérer les diffs, `search_repository`, `get_file_content` et `file_context` pour le *self-review agent*, et `add_merge_request_diff_note` pour publier les commentaires positionnés sur une ligne précise. La figure 4.1 illustre cette architecture.

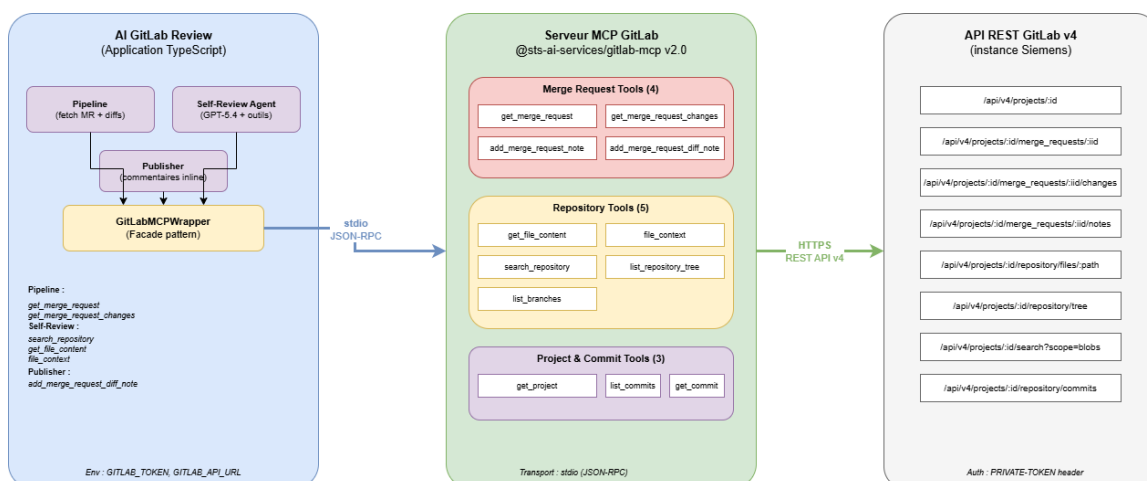


FIGURE 4.1 – Intégration MCP : le serveur GitLab expose ses outils via stdio, la facade `GitLabMCPWrapper` les consomme

4.2 Développements réalisés

4.2.1 Les agents spécialisés

Le pipeline de revue s'appuie sur quatre agents spécialisés, chacun dédié à un domaine d'analyse distinct. Chaque agent hérite de la classe abstraite `BaseAgent`, qui applique le *Template Method pattern* : la méthode `analyze()` orchestre trois étapes : construction du *prompt*, appel au LLM, puis extraction structurée de la réponse, tandis que les sous-

classes ne définissent que leur catégorie et leur *prompt* Markdown dédié (stocké dans `src/prompts/`).

```
export abstract class BaseAgent {
  public abstract readonly category: string;

  /**
   * Analyze a diff payload.
   * @param _input Diff payload.
   * @returns Empty findings for the scaffold stage.
   */
  public analyze(_input: string): Promise<ReviewFinding[]> {
    return Promise.resolve([]);
  }
}
```

FIGURE 4.2 – Classe abstraite `BaseAgent` – *Template Method pattern*

Les quatre agents concrets sont nommés par domaine métier, et non par technologie :

- `SecurityAgent` : vulnérabilités OWASP, secrets exposés, authentification, injections, XSS/CSRF;
- `LogicDefectsAgent` : dérérérencements nuls, `await` manquants, retours incorrects, branches mortes;
- `ErrorHandlingAgent` : `catch` vides, promesses avalées, échecs silencieux;
- `CodeQualityAgent` : *magic strings*, code mort, duplication.

Chaque agent concret se réduit à une déclaration de catégorie :

```
import { BaseAgent } from "../base-agent";
import { FINDING_TYPE } from "../config/constants/findings";

You, 3 weeks ago | 1 author (You)
export class SecurityAgent extends BaseAgent {
  public readonly category = FINDING_TYPE.SECURITY;
}
```

FIGURE 4.3 – Exemple d'agent concret : `SecurityAgent`

L'instanciation repose sur le *Factory Method pattern* (`createReviewAgent()`) et un registre clé-valeur (`AgentRegistry`, accès $O(1)$) :

```
export function createReviewAgent(category: FindingCategory): BaseAgent {  
  if (category === FINDING_TYPE.SECURITY) {  
    return new SecurityAgent();  
  }  
  
  if (category === FINDING_TYPE.LOGIC_DEFECT) {  
    return new LogicDefectsAgent();  
  }  
  
  if (category === FINDING_TYPE.ERROR_HANDLING) {  
    return new ErrorHandlingAgent();  
  }  
  
  if (category === FINDING_TYPE.CODE_QUALITY) {  
    return new CodeQualityAgent();  
  }  
  
  throw new Error(`No agent registered for category: ${category}`);  
}
```

FIGURE 4.4 – *Factory Method* – création d'agents par catégorie

Chaque agent ne fait que de la détection : la vérification est déléguée au *self-review* (section 3.3.6), la génération de correctifs au *CodegenAgent*. Les catégories sont définies comme constantes typées :

```
export const FINDING_TYPE = {  
  SECURITY: "security",  
  LOGIC_DEFECT: "logic-defect",  
  ERROR_HANDLING: "error-handling",  
  CODE_QUALITY: "code-quality"  
} as const;
```

FIGURE 4.5 – Constantes de catégories – `as const` pour typage littéral

L'orchestrateur constitue le point d'entrée du pipeline. La figure 4.6 montre la méthode `orchestrate()`, qui construit le *StateGraph* décrit en section 3.3.1, le compile et l'exécute.

```

/**
 * Execute the full code review workflow.
 * @param input - Review input containing project and MR IDs
 * @returns Complete review output with findings
 */
async orchestrate(input: ReviewInput): Promise<ReviewOutput> {
  const workflow = new StateGraph(OrchestratorAnnotation)
    .addNode(ORCHESTRATOR_NODES.FETCH_MR, this.fetchMergeRequest.bind(this))
    .addNode(ORCHESTRATOR_NODES.FETCH_DIFFS, this.fetchDiffs.bind(this))
    .addNode(ORCHESTRATOR_NODES.ANALYZE_CONTEXT, this.analyzeProjectContext.bind(this))
    .addNode(ORCHESTRATOR_NODES.CLASSIFY_DIFFS, this.classifyDiffs.bind(this))
    .addNode(ORCHESTRATOR_NODES.COORDINATE_AGENTS, this.coordinateAgents.bind(this))
    .addNode(ORCHESTRATOR_NODES.FINALIZE, this.finalize.bind(this))
    .addEdge(ORCHESTRATOR_STEPS.START, ORCHESTRATOR_NODES.FETCH_MR)
    .addEdge(ORCHESTRATOR_NODES.FETCH_MR, ORCHESTRATOR_NODES.FETCH_DIFFS)
    .addEdge(ORCHESTRATOR_NODES.FETCH_DIFFS, ORCHESTRATOR_NODES.ANALYZE_CONTEXT)
    .addEdge(ORCHESTRATOR_NODES.ANALYZE_CONTEXT, ORCHESTRATOR_NODES.CLASSIFY_DIFFS)
    .addEdge(ORCHESTRATOR_NODES.CLASSIFY_DIFFS, ORCHESTRATOR_NODES.COORDINATE_AGENTS)
    .addEdge(ORCHESTRATOR_NODES.COORDINATE_AGENTS, ORCHESTRATOR_NODES.FINALIZE)
    .addEdge(ORCHESTRATOR_NODES.FINALIZE, ORCHESTRATOR_STEPS.END);
  const app = workflow.compile();
  const result = await app.invoke({ input });
  if (!result.synthesizedOutput) {
    throw new Error(ORCHESTRATOR_MESSAGES.FAILED_SYNTHESIZED_OUTPUT);
  }
  return result.synthesizedOutput;
}

```

FIGURE 4.6 – Méthode `orchestrate()` – construction du `StateGraph` `LangGraph` à six nœuds

Le nœud `coordinateAgents` utilise `Send` de `LangGraph` pour distribuer les hunks aux agents en parallèle, puis `LangGraph` regroupe les findings avant le nœud `finalize`.

Prompts Markdown

Chaque agent dispose d'un *prompt* dédié stocké dans `src/prompts/` en Markdown. Le chargement est assuré par `PromptLoader` :

```

const PROMPT_ROOT = join(process.cwd(), "src", "prompts");
...
export class PromptLoader {
  /**
   * Load a markdown prompt by name.
   * @param promptName Prompt file name without extension.
   * @returns Prompt content.
   */
  public static load(promptName: string): string {
    const filePath = join(PROMPT_ROOT, `${promptName}.md`);
    return readFileSync(filePath, "utf8");
  }
}

```

FIGURE 4.7 – `PromptLoader` – chargement des prompts Markdown

Le contenu dynamique (titre de la [MR](#), diff) est transmis dans le *user message*, pas dans le *prompt* système : les prompts restent des fichiers statiques versionnés. Tous suivent une structure commune détaillée dans le tableau 4.3 :

TABLE 4.3 – Structure interne du prompt `classifier.md`

Section	Rôle
# Identity	Rôle et mission du modèle
# Pipeline Context	Position dans le pipeline (étape 1/3)
# Task	Consigne principale
# Categories	4 catégories avec priorité et <i>KEY SIGNAL</i>
# Skip Criteria	Conditions pour marquer un <i>hunk</i> propre
# File-Type Awareness	Règles spécifiques par type de fichier
# Recall Bias Rule	Biais explicite : en cas de doute, router
# Cross-Hunk Awareness	Relations entre <i>hunks</i> d'un même fichier
# Reasoning Protocol	<i>Chain-of-thought</i> imposé (analyse → décision)
# Common False Positives	Anti-hallucination : patterns bénins à ignorer
# Boundary Examples	4+ <i>few-shot</i> en <i>unified diff</i> réel (~30% propres)
# Output Format	Aligné avec le schéma Zod

Le champ `analysis` est placé avant `category` dans le schéma de sortie, forçant le modèle à décrire factuellement le changement *avant* de le classifier (*chain-of-thought*).

4.2.2 Le classificateur de diffs

Le classificateur est le composant le plus critique du pipeline : un faux négatif ici signifie qu'un vrai problème ne sera jamais détecté. Le biais est donc explicitement orienté vers le *recall* : en cas de doute, router (un faux positif ne coûte que ~\$0,002).

Le `DiffParser` découpe chaque diff en `HunkContext[]`. Chaque `HunkContext` porte les métadonnées (fichier, ligne, index) que le `LLM` ne voit jamais : seul le texte brut du hunk lui est transmis (pattern *Hunk Isolation*). Le `DiffClassifier` envoie ensuite les hunks au `LLM` avec le titre et la description de la [MR](#). La sortie est un `HunkClassification[]` validé par Zod. Chaque hunk reçoit une catégorie unique (priorité : security > logic-defect > error-handling > code-quality) ou est marqué `hasIssue = false` s'il ne contient que des changements non logiques.

Le schéma Zod impose un ordre de champs qui force le *chain-of-thought* : `analysis` → `hasIssue` → `category` → `confidence` → `reasoning`, avec deux contraintes croisées `.refine()` pour garantir la cohérence.

4.2.3 La déduplication sémantique

Même avec un classificateur *per-hunk*, des doublons peuvent apparaître : hunks adjacents déclenchant des findings quasi identiques, ou même pattern répété dans plusieurs fichiers. Le module de déduplication élimine ces redondances en trois couches (pattern *Facade*) :

1. **SimilarityCalculator** : similarité cosinus entre embeddings (**text-embedding-3-large**, seuil 0,50) ;
2. **ClusteringEngine** : regroupement des findings similaires en clusters de 2 à 8 éléments (pattern *Strategy*) ;
3. **SemanticDeduplicator** : façade orchestrant embeddings → similarité → clustering → sélection du finding représentant (**trustScore** le plus élevé).

L'approche est hybride : la similarité cosinus filtre rapidement, puis un second seuil LLM (0,55) affine les cas limites.

4.2.4 Les outils de recherche pour agents

Les agents de détection n'utilisent aucun outil : ils analysent le diff seul. Le *self-review agent* dispose en revanche de trois outils pour vérifier les findings incertains contre le codebase réel :

1. **search_code** : recherche par mot-clé dans le dépôt (ex. : vérifier l'existence d'une sanitisation en amont) ;
2. **get_file** : récupération du contenu complet d'un fichier ;
3. **read_file_lines** : lecture d'une plage de lignes spécifique, optimisant les tokens consommés.

Chaque outil est déclaré avec le helper `tool()` de LangChain v1 et un schéma Zod validant les arguments. Le nombre d'appels est limité par le middleware `modelCallLimitMiddleware` de LangGraph.

4.2.5 Les schémas Zod de *structured output*

Trois schémas Zod sont définis dans `src/types/schemas/` :

- **ReviewFindingSchema** : quatre champs (`title`, `description`, `codeSnippet?`, `trustScore`). Les métadonnées de positionnement (fichier, ligne) sont injectées par le runtime, pas par le LLM.
- **HunkClassificationSchema** : sept champs ordonnés pour le *chain-of-thought*, avec deux contraintes `.refine()`.
- **DiffClassificationSchema** : enveloppe un tableau de **HunkClassification**.

Les types TypeScript sont dérivés par `z.infer<typeof Schema>` : une seule source de vérité, zéro *drift*. Chaque champ utilise `.describe()` comme micro-instruction pour guider le modèle.

4.3 Tracing et monitoring

4.3.1 LangSmith : tracing des appels LLM

LangSmith est la plateforme de tracing native de LangChain. L'intégration se résume à une variable d'environnement `LANGCHAIN_TRACING_V2=true` : chaque appel LLM, chaque invocation d'outil et chaque transition LangGraph sont automatiquement capturés en tant que *spans* hiérarchiques. En production, cela permet de diagnostiquer les réponses inattendues d'un agent en inspectant le prompt exact reçu par le modèle, les tokens consommés et la latence par nœud.

4.3.2 Logging diagnostique

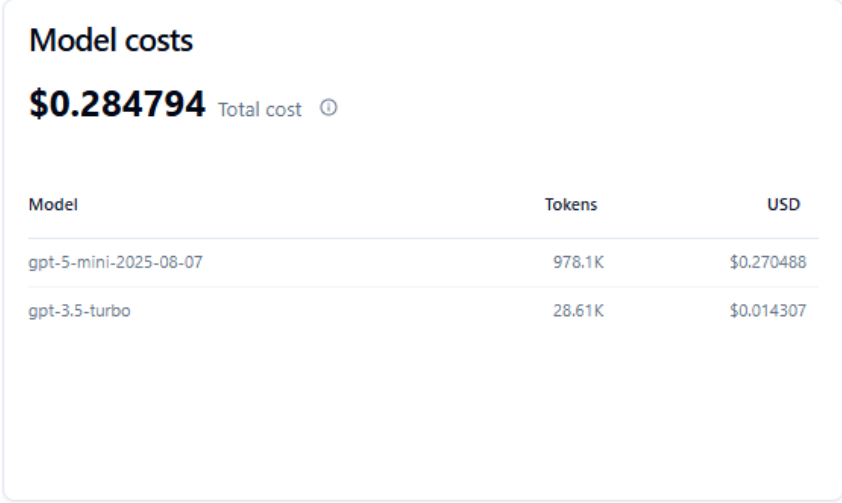
Chaque revue produit un log structuré en trois temps : au démarrage (variables d'environnement, version du modèle, projet cible), pendant l'exécution (nombre de hunks classifiés, findings par agent, durée par nœud), et en fin de pipeline (nombre de findings publiés, tokens totaux, coût estimé). Les messages d'erreur incluent le contexte suffisant pour reproduire le problème (ID de la [MR](#), fichier concerné, réponse brute du [LLM](#) si applicable).

4.3.3 Langfuse : tracing complémentaire

Langfuse est une plateforme open-source de tracing [LLM](#), intégrée en complément de LangSmith. LangSmith est la solution native de LangChain, utilisée pour le débogage des chaînes d'appels ; Langfuse, auto-hébergeable, offre un tableau de bord de coûts plus lisible et une meilleure vue d'ensemble par modèle. Les deux sont activés en parallèle via des variables d'environnement distinctes. Langfuse apporte trois vues utiles au suivi du pipeline : une vue tabulaire des traces avec durée et coût (figure 4.8), un tableau de bord des coûts ventilés par modèle (figure 4.9), et l'inspection détaillée de chaque appel d'outil dans une trace (figure 4.10).

☐	Timestamp ▼	Name	Input	Output	Observation Levels	Latency	Tok ^a
☐	☆ 2026-04-23 09:46:42	mr-review	["projectId":"28176","mergeRequestId":"31","focusAreas":["security"],"performa...		🔴 1 🟢 176	1m 31s	640,
☐	☆ 2026-04-23 09:43:12	mr-review	["projectId":"28176","mergeRequestId":"31","focusAreas":["security"],"performa...	["findingsCount":8,"summary":"8 findings"]	🔴 1 🟢 231	3m 6s	835,

FIGURE 4.8 – Langfuse : vue des traces de deux revues (durée, coût, tokens)



Model costs

\$0.284794 Total cost ⓘ

Model	Tokens	USD
gpt-5-mini-2025-08-07	978.1K	\$0.270488
gpt-3.5-turbo	28.61K	\$0.014307

FIGURE 4.9 – Langfuse : coût par modèle (GPT-5-mini \$0,27 – détection, GPT-3.5-turbo \$0,01 – ancien modèle de classification)



FIGURE 4.10 – Langfuse : trace d'un appel `search_code` par le *self-review agent*

4.4 Tests et validation

4.4.1 Stratégie de test

L'outil s'appuie sur **Vitest** comme framework de tests unitaires. Vitest est natif TypeScript, compatible ESM, et s'intègre directement avec le système de modules du projet sans configuration supplémentaire.

Les dépendances externes (appels LLM, serveur MCP, API GitLab) sont systématiquement mockées via `vi.fn()`. Aucun test ne déclenche de vrai appel réseau : les coûts restent nuls et les résultats déterministes.

En complément, AI Reviewer s'analyse lui-même dans la CI : chaque MR du projet est soumise à l'outil avant d'être fusionnée. Cette pratique de *dogfooding* permet de détecter

les régressions du pipeline sur un codebase familier et fournit un retour concret à chaque itération.

4.4.2 Tests unitaires (Vitest)

Cinq fichiers de tests couvrent les modules critiques du pipeline (tableau 4.4). L'ensemble totalise 47 tests, tous passants, exécutés en moins de 2 secondes.

TABLE 4.4 – Fichiers de tests et modules couverts

Fichier de test	Tests	Module testé
diff-classifier.test.ts	24	Schémas Zod du classificateur, regroupement par catégorie
openai-factory.test.ts	13	Factory Azure OpenAI, <i>model tiering</i> , températures
reviewer-agent.test.ts	3	Point d'entrée <code>AIGitLabReviewer</code> , publication
config-validator.test.ts	4	Lecture des variables d'environnement, config runtime
error-helpers.test.ts	3	Helpers de gestion d'erreurs

4.4.3 Couverture et parties non couvertes

La couverture globale est de **22 %** en lignes. Ce chiffre brut inclut des fichiers hors périmètre (scripts, librairies embarquées). Trois zones restent volontairement non testées :

- **Orchestrateur LangGraph** : dépend du runtime LangGraph (`StateGraph`, `Send`), difficile à isoler sans test d'intégration.
- **Agents de détection** : la logique réside dans les prompts Markdown, pas dans le code TypeScript. Tester le code revient à vérifier un appel **LLM** mocké donc peu de valeur ajoutée.
- **GitLabMCPWrapper** : façade I/O. Chaque méthode transmet un appel au serveur **MCP** et retourne le résultat, sans logique métier à valider.

4.4.4 Validation fonctionnelle

La validation repose sur l'usage en production : AI Reviewer analyse les **MR** des quatorze équipes utilisatrices depuis la version 1.0. Les retours sont collectés via le channel Teams dédié (section 1.4). Trois indicateurs sont suivis :

- **Pertinence** : les commentaires sont-ils utiles au relecteur ? Les retours sont majoritairement positifs depuis la version 1.2 (classification *per-hunk* + *trust scoring*).
- **Faux positifs** : les commentaires non pertinents ont diminué à chaque version. Le *self-review* (version 1.3) a réduit les signalements sur ce point.
- **Latence** : une revue complète s'exécute en cinq minutes en moyenne, compatible avec un pipeline **CI**.

4.4.5 Limites connues

Trois limites méritent d'être mentionnées. Premièrement, le pipeline repose sur des LLM qui peuvent halluciner : un agent peut signaler un bug inexistant avec un raisonnement plausible. Le *trust scoring* et le *self-review* réduisent ce risque mais ne l'éliminent pas. Si un développeur perd confiance à cause d'un faux positif bien formulé, l'adoption de l'outil peut reculer. Deuxièmement, le dataset d'évaluation (AACR-Bench) est partiellement annoté par LLM, ce qui introduit un biais circulaire : on évalue des modèles avec des données produites par des modèles similaires. Le dataset interne prévu devrait atténuer cette limite. Troisièmement, le code analysé peut contenir des secrets ou des données sensibles dans les diffs : ces informations transitent par Azure OpenAI, ce qui reste dans le périmètre approuvé par Siemens, mais elles apparaissent dans les traces LangSmith et Langfuse hébergées en externe. Une politique de rétention courte est appliquée pour limiter l'exposition.

4.5 Résultats et perspectives

4.5.1 Résultats quantitatifs

L'outil a été évalué sur un dataset issu d'AACR-Bench (LIANG et al. 2026), un benchmark open source publié par Alibaba sur GitHub. Ce dataset contient 200 *pull requests* réelles issues de 50 projets open source, avec 2 145 commentaires de revue annotés par des experts et enrichis par LLM. Les annotations originales utilisent quatre catégories (*Security*, *Defect*, *Maintainability*, *Performance*) qui ne correspondent pas exactement aux quatre catégories d'AI Reviewer (sécurité, défauts logiques, gestion d'erreurs, qualité de code). J'ai donc ré-annoté le dataset en deux passes : une première passe automatisée avec GitHub Copilot pour convertir les catégories et reformater les entrées, puis une relecture manuelle pour corriger les cas ambigus et valider la cohérence des labels.

La figure 4.11 présente les métriques globales obtenues par trois modèles Azure OpenAI sur ce dataset :

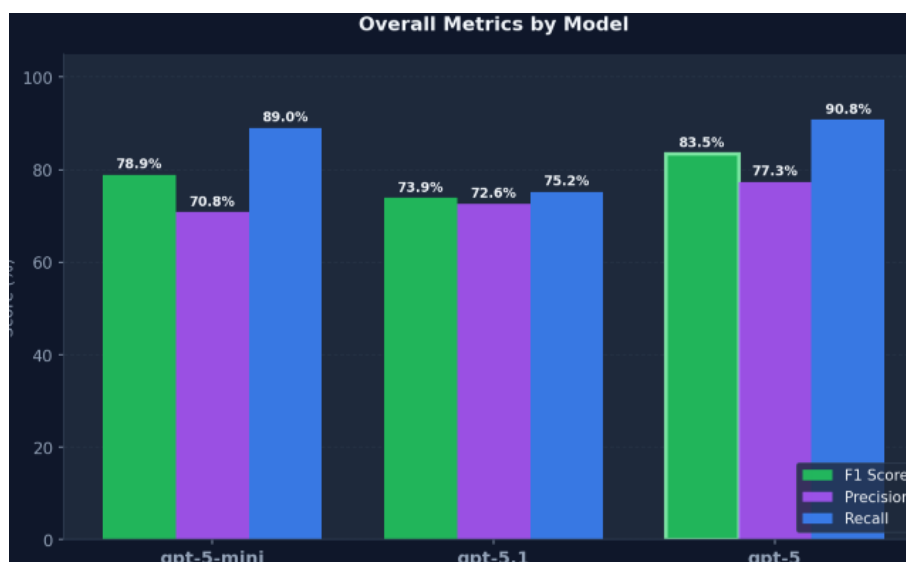


FIGURE 4.11 – Métriques F1, précision et rappel par modèle sur le dataset de validation

GPT-5 Mini, utilisé pour les agents de détection, atteint un F1 de 78,9 % avec un rappel de 83,5 % et une précision de 70,8 %. Ce résultat est cohérent avec le biais *recall* > *precision* voulu par le pipeline : près d'un finding sur trois est un faux positif au stade de la détection, ce qui justifie l'existence du *self-review* en aval pour les filtrer. GPT-5.1 obtient le meilleur F1 (89,0 %) avec une précision de 90,8 %, ce qui en fait le candidat naturel pour le *self-review*. GPT-5 se situe entre les deux (F1 de 75,2 %).

Sur le plan opérationnel, l'outil est aujourd'hui déployé auprès de quatorze équipes de développement (103 utilisateurs actifs). Une revue s'exécute en cinq minutes en moyenne, un temps variable selon le modèle utilisé et la taille de la [MR](#).

Note : le benchmark AACR-Bench a été exécuté sur les modèles GPT-5, GPT-5 Mini et GPT-5.1, disponibles sur Azure OpenAI au moment de l'évaluation. Le refactoring en cours migre vers les modèles GPT-5.4 et GPT-5.4 Mini, plus récents. La capture Langfuse (figure 4.9) montre également GPT-3.5-turbo, un ancien modèle de classification remplacé depuis.

4.5.2 Perspectives

La migration en cours (section 4.1.2) finalise la migration vers LangGraph 1.3 et les modèles GPT-5.4, qui devraient améliorer le déterminisme des analyses et l'intégration des outils par le *self-review agent*.

Au-delà de cette migration, plusieurs améliorations sont identifiées pour les prochaines itérations : constitution d'un dataset annoté de [MR](#) Siemens réelles, ajout d'agents dédiés au style et aux fautes de frappe, agent personnalisable par équipe, enrichissement du

contexte fourni aux agents, réponse interactive aux commentaires, intégrations Jira et Confluence, support Bitbucket, et poursuite de l'onboarding de nouvelles équipes.

Conclusion

Ce rapport a présenté AI GitLab Review, un outil d'analyse automatisée des *merge requests* conçu pour compléter la revue de code humaine chez Siemens Digital Industries Software. Partant d'un prototype monolithique à agent unique, le projet a évolué en sept mois vers une architecture multi-agents orchestrée par LangGraph : quatre agents spécialisés (sécurité, défauts logiques, gestion d'erreurs, qualité de code), un classificateur *per-hunk*, un mécanisme de *trust scoring* avec *self-review*, une déduplication par similarité sémantique, et une publication des commentaires via le Model Context Protocol. L'outil est aujourd'hui utilisé par quatorze équipes, soit 103 développeurs.

Bilan technique. La question posée en introduction " concevoir un système d'agents capable de compléter la revue humaine tout en maintenant un taux de faux positifs acceptable " a trouvé une réponse satisfaisante. Le classificateur *per-hunk* et le *trust scoring* à trois paliers ont été les deux décisions les plus déterminantes : le premier a résolu le problème de granularité, le second a réduit les faux positifs en réservant un modèle plus puissant aux findings incertains. L'évaluation sur AACR-Bench confirme un F1 de 78,9 % pour le modèle de détection, avec un rappel de 83,5 % cohérent avec le biais voulu.

Bilan personnel. Ce projet m'a fait progresser sur plusieurs plans. Sur le plan technique, j'ai acquis des compétences en TypeScript, en architecture logicielle (*design patterns*, séparation des responsabilités), en *prompt engineering*, en orchestration d'agents avec LangGraph et en intégration d'API (Azure OpenAI, MCP, LangSmith). Sur le plan professionnel, j'ai appris à mener un projet de bout en bout : de la conception à la mise en production, en passant par le support utilisateur et la communication avec les équipes. Le déploiement auprès de de plusisuers equipes : chaque équipe avait ses conventions de codage, sa tolérance aux faux positifs et ses attentes en matière de format de commentaire. J'ai appris à prioriser les demandes et à communiquer les limites de l'outil sans décourager l'adoption.

La principale difficulté technique a été le non-déterminisme des LLM : chaque modification de prompt pouvait améliorer un cas et en dégrader un autre, ce qui rendait l'évaluation

lente et parfois frustrante. Un moment particulièrement bloquant a été la phase de classification *per-file* (version 1.1) : le classificateur regroupait des hunks indépendants sous une même catégorie, ce qui produisait des findings incohérents. Deux semaines de tentatives d'amélioration des prompts n'ont rien donné ; c'est le passage à la classification *per-hunk* qui a résolu le problème en une itération. J'ai appris à structurer les prompts de manière défensive (exemples *few-shot*, anti-hallucination, *chain-of-thought* imposé) pour réduire cette variabilité.

Ouverture. AI Reviewer continuera d'évoluer au-delà de mon implication directe. Au moment de la rédaction de ce rapport, je m'apprête à être affecté à de nouveaux projets de R&D en outils agentiques, toujours au sein de l'équipe AIS. Cette première année m'a appris à mener un projet technique de bout en bout ; la deuxième année d'alternance, jusqu'en septembre 2027, sera l'occasion d'élargir mon champ d'action et de découvrir d'autres applications de l'intelligence artificielle à l'ingénierie logicielle.

Liste des acronymes

API Application Programming Interface. [12](#), [13](#), [21](#), [25](#), [27](#), [34](#), [39](#)

BDIA Bases de Données et Intelligence Artificielle. [1](#)

CD Continuous Delivery. [8](#), [11](#), [24](#)

CI Continuous Integration. [8](#), [11](#), [13](#), [16](#), [20](#), [24](#), [34](#), [35](#)

IA Intelligence Artificielle. [1](#), [10](#), [11](#), [15](#), [27](#)

LLM Large Language Model. [1](#), [13](#), [14](#), [18](#), [20](#), [22](#), [25](#), [26](#), [31–36](#), [39](#)

MCP Model Context Protocol. [12](#), [16–18](#), [20](#), [21](#), [25–27](#), [34](#), [35](#), [39](#)

MR Merge Request. [11–13](#), [15](#), [18](#), [20](#), [23](#), [24](#), [31](#), [33–35](#), [37](#)

REST Representational State Transfer. [27](#)

Références bibliographiques

- ANTHROPIC (2025). *Claude Code Review – Automated PR Reviews with Multi-Agent Analysis*. URL : <https://code.claude.com/docs/en/code-review> (visité le 10/05/2026).
- COHEN, Jason (2006). *Best Kept Secrets of Peer Code Review*. Austin, TX : SmartBear Software.
- CREWAI, INC. (2024). *CrewAI – Framework for Orchestrating AI Agents*. URL : <https://docs.crewai.com/> (visité le 10/05/2026).
- GITHUB, INC. (2026). *GitHub Copilot SDK – Multi-platform SDK for Integrating GitHub Copilot Agent*. URL : <https://github.com/github/copilot-sdk> (visité le 10/05/2026).
- LANGCHAIN, INC. (2024a). *LangChain.js – Documentation*. URL : <https://js.langchain.com/docs/introduction/> (visité le 10/05/2026).
- (2024b). *LangGraph – Build Stateful AI Agents*. URL : <https://langchain-ai.github.io/langgraphjs/> (visité le 10/05/2026).
 - (2024c). *LangSmith – LLM Observability Platform*. URL : <https://docs.smith.langchain.com/> (visité le 10/05/2026).
- LIANG, Wentao et al. (2026). *AACR-Bench : An Automated Benchmark for Code Review*. URL : <https://github.com/ZJU-ACES-ISE/AACR-Bench> (visité le 10/05/2026).
- MCDONNELL, Colin (2024). *Zod – TypeScript-first Schema Validation*. URL : <https://zod.dev/> (visité le 10/05/2026).
- MICROSOFT (2024). *Azure OpenAI Service – Documentation*. URL : <https://learn.microsoft.com/en-us/azure/ai-services/openai/> (visité le 10/05/2026).
- SHORTINO, Paul (2020). *Conventional Comments*. URL : <https://conventionalcomments.org/> (visité le 10/05/2026).